Project No: A2285-251

Hardware-Software Co-Design For Large Language Model

Submitted by: Yap Chen Kit

Supervisor: Professor Chang Chip Hong

Co-supervisor: Koca Nazim Altar

School of Electrical and Electronic Engineering

A final year project report presented to Nanyang Technological University
in partial fulfilment of the requirements for the
Degree of Bachelor of Engineering

2026

Contents

Abstract iv

Acknowledgement v

Acronyms vi

Symbols vii

List of Figures vii

List of Tables viii

1 Introduction 1

1.1 Motivations 1

1.2 Objectives and Scope 2

1.3 Organisation of Report 2

2 Literature Review 3

2.1 Transformer Model Architecture 3

2.2 TinyLlama Model Architecture 4

2.2.1 Token Embeddings 5

2.2.2 Rotary Positional Embedding 6

2.2.3 RMSNorm 7

2.2.4 Grouped-Query Self-Attention 7

2.2.5 Softmax 9

2.2.6 SwiGLU Feed-Forward Network 10

2.3 Efficient Attention Mechanisms 11

2.3.1 Sparse Transformer 11

2.3.2 BlockBERT 12

2.3.3 Performer 13

2.3.4 Linear Transformer 13

2.4 FPGA Acceleration of Neural Networks 14

2.5 Quantisation for Efficient Inference 14

2.6 HLS-Based Hardware Design 14

2.6.1 Pipelining and the Initiation Interval 15
2.6.2 Scaling Transformations: Unrolling and Dataflow 15
2.6.3 Memory Transformations 16

3 System Architecture and Design 17
3.1 Top-Level System Overview 17
3.2 Hardware/Software Partitioning 18
3.3 Memory Architecture 19
3.4 Quantisation Scheme 21

4 Implementation 23
4.1 RMSNorm 24
4.1.1 Implementation 24
4.1.2 C-Simulation Results 25
4.1.3 HLS Synthesis Results 26
4.2 Quantisation 28
4.2.1 Implementation 28
4.2.2 C-Simulation Results 30
4.2.3 HLS Synthesis Results 31
4.3 Dequantisation 33
4.3.1 Implementation 33
4.3.2 C-Simulation Results 33
4.3.3 HLS Synthesis Results 34
4.4 Matrix–Vector Multiplication 36
4.4.1 Implementation 36
4.4.2 C-Simulation Results 38
4.4.3 HLS Synthesis Results 39
4.5 Softmax 42
4.5.1 Implementation 42
4.5.2 C-Simulation Results 42
4.5.3 HLS Synthesis Results 42
4.6 Grouped-Query Attention 42
4.6.1 Baseline: Floating-Point Attention 42
4.6.2 Refined: Per-Head INT8 Attention 42
4.6.3 C-Simulation Results 42
4.6.4 HLS Synthesis Results 42
4.7 SwiGLU Feed-Forward Network 42
4.7.1 Implementation 42

4.7.2 C-Simulation Results 42
4.7.3 HLS Synthesis Results 42
4.8 Top-Level Forward Function 42
4.8.1 AXI Interface and Buffer Layout 42
4.8.2 Layer Loop and KV Cache Management 42
4.8.3 Known Limitations and Design Decisions 42

5 Results and Discussion 43
5.1 Experimental Setup 43
5.2 HLS Synthesis Results 43
5.3 Functional Verification 44
5.4 Inference Performance 44
5.5 Energy Efficiency 44
5.6 Comparison Against Prior Work 45
5.7 Discussion 45

6 Conclusion and Future Work 46
6.1 Conclusion 46
6.2 Recommendations for Future Work 47

Reflection on Learning Outcome Attainment 48

Acknowledging/Declaring the Use of GAI 49

References 49

A HLS Kernel Source Code 50

B Vivado Synthesis and Implementation Reports 51

C Quantisation Calibration Data 52

Abstract

Small-scale large language models (LLMs) such as TinyLlama-1.1B have emerged as a compelling class of compact transformer models that deliver meaningful natural language understanding capabilities within a fraction of the memory footprint of frontier models such as GPT-4. Yet even at the billion-parameter scale, these models impose substantial computational demands that limit their deployment in embedded and edge environments.

Currently, these language models are mostly deployed on Graphics Procesing Units (GPU). However, GPU inference is energy-intensive and memory-bandwidth-bound, particularly during the autoregressive decode phase since the time and memory complexity of self-attention are quadratic in sequence length.

Field-programmable gate arrays (FPGAs) offer a compelling alternative to GPU-based inference by providing reconfigurable hardware with low power consumption and predictable latency. But mapping a billion-parameter transformer to resource-constrained FPGA fabric presents significant challenges in quantization, memory tiling, and hardware-software partitioning.

This project starts off with a comprehensive profiling of a RTX4060 Laptop GPU running a pytorch inference of the Llama 2-based TinyLlama 1.1B model from HuggingFace with default GPU kernel settings to observe for bottlenecks and to measure propotion of wall-clock time required by the self-attention and feed-forward layer during the forward pass. Based on the results, an FPGA-based accelerator is proposed to accelerate the architecture of the Tinyllama model. For this project, emphasis is placed on the hardware implementation and optimization of the self attention layer and the feed-forward layer.

This project implements and optimises the inference pipeline of TinyLlama-1.1B on the Xilinx ZCU102 FPGA platform using Vitis High-Level Synthesis (HLS). The core transformer components – including RMSNorm, matrix–vector multiplication, multi-head attention, and the SwiGLU feed-forward network – were implemented as quantised INT8 HLS kernels.

(The results demonstrate that FPGA acceleration with INT8 quantisation achieves competitive inference performance while operating within the ZCU102's resource budget, validating the viability of transformer-based language model deployment on reconfigurable hardware.)

Acknowledgement

I would like to express my sincere gratitude to Professor Chang Chip Hong for being my supervisor and giving me multiple chances to explore different research areas since my second year until my final year of undergraduate studies, and allowing me to pursue this topic as my FYP.

I would also like to thank my co-supervisor, Koca Nazim Altar, for supporting me during my final year project, in addition to his full-time job and his PhD tasks. His guidance have helped me overcome technical and mental roadblocks with his kindness and expertise. Altar have provided me valuable knowledge that helped bring this project to fruition.

Yap Chen Kit
May 2026

Acronyms

AXI	Advanced eXtensible Interface
BRAM	Block Random Access Memory
DDR	Double Data Rate (SDRAM)
DMA	Direct Memory Access
DSP	Digital Signal Processor
FIFO	First-In, First-Out
FPGA	Field-Programmable Gate Array
FF	Flip-Flop
GeLU	Gaussian Error Linear Unit
GPU	Graphics Processing Unit
HLS	High-Level Synthesis
II	Initiation Interval
INT8	8-bit Integer
IP	Intellectual Property (core)
LLM	Large Language Model
LUT	Look-Up Table
MHA	Multi-Head Attention
MLP	Multi-Layer Perceptron
NPU	Neural Processing Unit
PL	Programmable Logic
PS	Processing System
PTQ	Post-Training Quantisation
QKV	Query, Key, Value
ReLU	Rectified Linear Unit
RMSNorm	Root Mean Square Layer Normalisation
RTL	Register-Transfer Level
SoC	System-on-Chip
SwiGLU	Swish-Gated Linear Unit
XRT	Xilinx Runtime
ZCU102	Zynq UltraScale+ MPSoC Evaluation Board (ZCU102)

Symbols

Symbol	*Description*
d_{model}	Hidden dimension of the model (2048 for TinyLlama-1.1B)
d_{ff}	Feed-forward network hidden dimension
d_k	Dimension of each attention head
L	Number of transformer layers
H	Number of attention heads
V	Vocabulary size
T	Sequence length (number of tokens)
W_Q, W_K, W_V	Query, Key, Value projection weight matrices
W_O	Output projection weight matrix
Q, K, V	Query, Key, Value activation matrices
softmax	Softmax normalisation function
s	Quantisation scale factor
z	Quantisation zero-point offset
b	Number of quantisation bits
$\hat{x}$	Quantised representation of variable x
II	Initiation interval of a pipelined loop (clock cycles)
f_{clk}	Clock frequency (MHz)
N_{LUT}	Number of look-up tables consumed
N_{DSP}	Number of DSP blocks consumed
P	Power consumption (mW)

List of Figures

2.1 Connectivity matrices for full attention (a), Sparse Transformer with strided pattern (b), and Sparse Transformer with fixed pattern (c). In (b) and (c), full connectivity between all elements is preserved when the two attention heads are computed sequentially, while sparsity in each individual head leads to significantly reduced computation **PLACEHOLDER**. 12

2.2 Pipeline characteristics showing latency L, initiation interval I, and total cycle count $C = L + I(N-1)$ for N inputs **PLACEHOLDER**. . 15

2.3 Horizontal unrolling, vertical unrolling, and dataflow as means to increase parallelism. Rectangles represent buffer space such as registers or on-chip RAM. (a) Before any transformation. (b) Horizontal unrolling: four independent inputs processed in parallel. (c) Vertical unrolling: one input combined with multiple buffered values. (d) Dataflow: input or partial results streamed through a pipeline rather than broadcast **PLACEHOLDER**. 16

List of Tables

2.1 TinyLlama-1.1B architectural hyperparameters **PLACEHOLDER**. . 4

3.1 Hardware/software partitioning of the TinyLlama inference pipeline. Operations marked PL are executed in the programmable logic accelerator kernel; operations marked PS run on the ARM Cortex-A53. . . 19

3.2 Memory hierarchy of the TinyLlama inference accelerator on the ZCU102. Weight and KV cache sizes are after INT8 quantisation. On-chip buffers are reused across all 22 layers. 21

4.1 Prototype and full TinyLlama-1.1B configuration parameters used across kernel development. 23

4.2 C-simulation output for `rmsnorm` with input $\mathbf{x} = [1, -2, 3, -4, 5, -6, 7, -8]$ and weight $\mathbf{g} = [0.5, 1.0, 1.5, 2.0, 0.5, 1.0, 1.5, 2.0]$. All eight elements match the software reference exactly. 26

4.3 Timing summary for `rmsnorm_top` at the 150 MHz (6.667 ns) target clock. 27

4.4 Top-level latency summary for `rmsnorm_top` at 150 MHz. 27

4.5 Resource utilisation of the `rmsnorm` function synthesised at $S = 8$ on the ZCU102 (`xczu9eg-ffvb1156-2-e`). Figures in the core-only column exclude the AXI master adapter overhead, which is not present when the function is inlined into the full kernel. 27

4.6 Latency breakdown of `rmsnorm` ($S = 8$) by internal pipeline stage at 150 MHz. Total function latency is 144 cycles (960 ns). 27

4.7 C-simulation output for `quantize` with input $\mathbf{x} = [0.5, -1.2, 3.7, -0.3, 2.1, -4.5, 1.0, -2.8]$. All INT8 values and both scale factors match the software reference exactly. GS = 4 so group 0 covers indices 0–3 and group 1 covers indices 4–7. 31

4.8 Timing summary for `quantize_top` at 150 MHz. 31

4.9 Top-level latency summary for `quantize_top` at 150 MHz. 32

4.10 Resource utilisation of `quantize` synthesised at $S = 8$ on the ZCU102. The core-only column excludes the AXI master adapter. 32

4.11 Latency breakdown of `quantize` ($S = 8$, $GS = 4$) by internal pipeline stage at 150 MHz. Total function latency is 84 cycles (560 ns). 32
4.12 C-simulation output for `dequantize` with group scales $s_0 = 0.029134$ (indices 0–3) and $s_1 = 0.035433$ (indices 4–7). All eight dequantised float values match the software reference with zero absolute error. . . 34
4.13 Timing summary for `dequantize_top` at 150 MHz. 34
4.14 Top-level latency summary for `dequantize_top` at 150 MHz. 35
4.15 Resource utilisation of `dequantize` synthesised at $S = 8$ on the ZCU102. The core-only column excludes the AXI master adapter. 35
4.16 Latency breakdown of `dequantize` ($S = 8$, $GS = 4$) at 150 MHz. Total function latency is 45 cycles (300 ns). 35
4.17 C-simulation output for `matmul` with quantised input $\hat{\mathbf{x}} = [17, -41, 127, -10, 59, -127, 28, -79]$ (group scales $s_0 = 0.029134$, $s_1 = 0.035433$) and an 8×8 scaled identity weight matrix. All eight output elements match the software reference within tolerance. 39
4.18 Timing summary for `matmul_top` at the 150 MHz (6.667 ns) target clock. 39
4.19 Top-level latency summary for `matmul_top` at 150 MHz. 40
4.20 Resource utilisation of `matmul` synthesised at $N = D = 8$ on the ZCU102 (`xczu9eg-ffvb1156-2-e`). The core-only column excludes the AXI master adapter and S-AXILite control interface, which are not present when the function is inlined into the full kernel. 40
4.21 Latency breakdown of `matmul` ($N = D = 8$, $GS = 4$) at 150 MHz. Total function latency is 146 cycles (973 ns). 40

5.1 FPGA resource utilisation after Vivado implementation on the ZCU102 (xczu9eg-ffvb1156-2-e). Percentages are relative to total available resources on the device. 43
5.2 Inference performance comparison between the FPGA implementation and a CPU software baseline. Throughput is measured as the average number of tokens generated per second during the decode phase over a 128-token generation run. 44
5.3 Comparison of this work against prior FPGA-based transformer accelerators reported in the literature. 45

Chapter 1 Introduction

1.1 Motivations

Large language models (LLMs) have demonstrated excellent capabilities across a wide range of natural language understanding and generation tasks, but their deployment in resource-constrained environments remains a significant challenge **PLACEHOLDER**. State-of-the-art large language models (LLMs) such as GPT-4 and LLaMA require tens or hundreds of gigabytes of memory and specialised GPU clusters to achieve practical inference throughput, making them inaccessible for embedded, edge, and real-time applications.

The llama 2-based TinyLlama-1.1B model **PLACEHOLDER** represents a class of small-scale LLMs that seek to balance capability with deployability. With only 1.1 billion parameters, TinyLlama retains competitive performance on common benchmarks while fitting within a much smaller memory footprint. But even at this scale, running inference on a conventional CPU is far too slow for real-time use, and GPUs consume too much power for battery-operated or always-on deployments.

Field-programmable gate arrays (FPGAs) offer a compelling alternative. Unlike ASICs, FPGAs are reconfigurable. The hardware can be modified precisely to the computational patterns of a specific model, enabling high-throughput, low-latency inference at a fraction of the power cost of a GPU **PLACEHOLDER**.

There are two inefficiencies of GPU-based inference. First, autoregressive decode is memory-bandwidth-bound. Each decode step must reload the full model weights from DRAM to perform matrix–vector multiplications against a single token, yet profiling of TinyLlama-1.1B on an RTX 4060 Laptop GPU reveals low memory bandwidth utilisation as the remainder is lost to kernel launch overhead and memory access latency between hundreds of fine-grained CUDA kernel invocations per step. Second, the GPU runtime cannot fuse operations across transformer sub-layers. RMSNorm, matrix–vector multiplication, and activation functions are peformed as separate kernels, each incurring CPU-to-GPU scheduling latency and cache eviction between them.

An FPGA accelerator eliminates both inefficiencies simultaneously. The weight data flows directly through a statically scheduled pipeline without repeated DRAM round-trips, and the entire transformer sub-layer (normalisation, projection, and activation) can be implemented as a single fused datapath with no inter-kernel overhead.

1.2 Objectives and Scope

The general objective of this project is to implement an AI accelerator by implementing and optimizing the inference pipeline of TinyLlama-1.1B on the Xilinx ZCU102 evaluation board using Vitis HLS. Due to the complexity of the ZCU102 board, we begin our initial kernel design by targeting the KRIA KR260 FPGA board. Since the KRIA KR260 FPGA have insufficient resource to implement the TinyLlama model, we decided to start with a scaled-down version of the model's dimensions.

To understand the bottleneck of AI model inference which will be targeted for acceleration, literature review is performed and profiliers are used to obtain performance metrics from Nvidia's RTX4060 Laptop GPU. Then, a simple matrix multiplication kernel is first implemented with High-Level Synthesis (HLS). By the end of the project, a fused kernel consisting of the self-attention and feed-forward layer of the TinyLlama archietecture is implemented and synthesized on an FPGA.

1.3 Organisation of Report

The remainder of this report is organised as follows. Chapter 2 reviews the relevant literature on FPGA-based transformer acceleration, INT8 quantisation techniques, and related HLS optimisation strategies. Chapter 3 presents the overall system architecture and the design decisions made at the hardware level. Chapter 4 describes the detailed implementation of each HLS kernel and the software driver layer. Chapter 5 presents the experimental results including resource utilisation, performance benchmarks, and comparison against baseline systems. Chapter 6 draws conclusions and identifies directions for future work.

Chapter 2 Literature Review

2.1 Transformer Model Architecture

The transformer, introduced by Vaswani et al. **PLACEHOLDER**, departed from the recurrent architectures that had dominated sequence modelling by replacing recurrence entirely with a mechanism called self-attention. In the original formulation, the model consisted of two stacks. An encoder that reads the full input sequence in parallel, and a decoder that generates the output sequence one token at a time while attending to the encoder's representations.

For language modelling and text generation, the decoder stack alone is sufficient. In a decoder-only transformer, the model is trained to predict the next token given all previous tokens. At inference time, the model operates autoregressively, where it processes a prompt, generates one token, appends that token to its context, and repeats until the desired output length is reached. TinyLlama follows this decoder-only paradigm.

The flow of data through a single decoder-only forward pass proceeds as follows. An input sequence of tokens is first mapped to continuous vector representations by an embedding lookup, yielding a matrix $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$ where T is the sequence length and d_{model} is the hidden dimension. Positional information is then injected, either through a fixed sinusoidal encoding as in the original transformer or through a learned rotary scheme as used in TinyLlama. This embedding matrix is then passed through L identical transformer layers stacked in sequence. Within each layer, the representation first passes through a normalisation step, then through a multi-head self-attention (MHA) sub-layer, with a residual connection adding the layer's input back to its output. The result is normalised again and passed through a feed-forward network (FFN) sub-layer, again with a residual connection. After all L layers, the final hidden states are projected to the vocabulary dimension by a linear layer called the language model (LM) head, and a softmax function converts the resulting logits to a probability distribution over the vocabulary. The token with the highest probability (or a sampled token) becomes the next output.

The self-attention mechanism is what gives the transformer its expressive power, but it also creates the dominant challenge for hardware accelerators. During the prefill phase, when the model processes the full input prompt in one pass, the computation is

dense and GPU-friendly, as all sequence positions are processed in parallel, achieving high arithmetic throughput. During the autoregressive decode phase, the model processes a single new token per step while attending over all previously generated tokens stored in a key–value (KV) cache. This single-token operation is memory-bandwidth-bound rather than compute-bound. The GPU must load the full model weight matrices from DRAM on every step despite performing only a small fraction of the arithmetic those weights could theoretically support.

2.2 TinyLlama Model Architecture

TinyLlama-1.1B **PLACEHOLDER** is a compact, open-source, decoder-only language model developed by Zhang et al. at the Singapore University of Technology and Design. Despite its relatively small parameter count of 1.1 billion, TinyLlama achieves competitive performance on commonsense reasoning and problem-solving benchmarks.

TinyLlama adopts the LLaMA-2 architecture **PLACEHOLDER** with several modifications over the original transformer. Table 2.1 summarises the key architectural hyperparameters. The model uses a hidden dimension $d_{\text{model}} = 2048$, an intermediate feed-forward dimension of 5632, 32 transformer layers, and a vocabulary of 32,000 tokens. Four architectural choices distinguish TinyLlama from the vanilla transformer described in section 2.1: rotary positional embeddings (RoPE), pre-normalisation with RMSNorm, the SwiGLU activation function in the feed-forward sub-layer, and grouped-query attention (GQA). Each of these is briefly introduced here and examined in detail in the subsections that follow.

Hyperparameter	Value
Parameters	1.1B
Hidden dimension d_{model}	2,048
FFN intermediate dim	5,632
Number of layers L	22
Query heads	32
KV head groups (GQA)	4
Context length	2,048 tokens
Vocabulary size	32,000

Table 2.1: TinyLlama-1.1B architectural hyperparameters **PLACEHOLDER**.

Rotary Positional Embedding (RoPE). Unlike the absolute sinusoidal positional encodings added to the input embeddings in the original transformer, RoPE **PLACEHOLDER**

encodes positional information by rotating the query and key vectors in the attention computation.

Pre-normalisation with RMSNorm. The original transformer applied layer normalisation after each sub-layer (post-norm). TinyLlama, following LLaMA-2, applies normalisation before each sub-layer (pre-norm). The normalisation function used is RMSNorm **PLACEHOLDER**, a simplified variant of LayerNorm that omits the mean-centring step, reducing the number of passes over the input vector and lowering hardware implementation complexity.

SwiGLU Feed-Forward Network. The feed-forward sub-layer in TinyLlama replaces the standard two-layer ReLU MLP with the SwiGLU variant **PLACEHOLDER**, which introduces a gating mechanism. The input is projected into two parallel streams, one of which is passed through the Swish activation function and then element-wise multiplied with the other.

Grouped-Query Attention (GQA). Standard multi-head attention computes separate key and value projections for each of the 32 query heads, resulting in a KV cache that grows as $O(L \cdot H \cdot T \cdot d_k)$ with sequence length T. TinyLlama reduces this overhead by sharing key and value representations across groups of query heads, using only 4 KV head groups for 32 query heads **PLACEHOLDER**. This reduces the KV cache size by a factor of 8, which is significant for memory- constrained FPGA deployment.

2.2.1 Token Embeddings

The first operation in every forward pass converts discrete token indices into continuous vector representations. The model maintains an embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d_{\text{model}}}$, where V is the vocabulary size and d_{model} is the hidden dimension. For TinyLlama, $V = 32{,}000$ and $d_{\text{model}} = 2{,}048$. Given an input sequence of T token indices $[t_1, t_2, \ldots, t_T]$, the embedding lookup produces:

$$\mathbf{X} = \begin{bmatrix} \mathbf{e}_{t_1} \\ \mathbf{e}_{t_2} \\ \vdots \\ \mathbf{e}_{t_T} \end{bmatrix} \in \mathbb{R}^{T \times d_{\text{model}}} \tag{2.1}$$

where $\mathbf{e}_{t_i}$ denotes the t_i-th row of $\mathbf{E}$. Each row is a d_{model}-dimensional dense vector learned during pretraining that positions semantically similar tokens close together in the embedding space.

Space complexity. The embedding matrix requires $V \times d_{\text{model}}$ parameters. In FP32 this amounts to $32{,}000 \times 2{,}048 \times 4 = 262\,\text{MB}$. This is one of the larger weight tensors in the model and is a candidate for quantisation since its access pattern during inference,

consisting of one row lookup per generated token, is entirely memory-bound.

Time complexity. The lookup itself is $\mathcal{O}(T)$ since it is a table index operation with no arithmetic; however, the subsequent layers treat $\mathbf{X}$ as a dense matrix and all downstream costs scale with $T \times d_{\text{model}}$.

2.2.2 Rotary Positional Embedding

Because the self-attention mechanism is permutation-invariant, treating the input as a set of vectors with no inherent order, the model must be given some means of distinguishing position m from position n. TinyLlama uses Rotary Positional Embedding (RoPE) **PLACEHOLDER**, which encodes position by rotating the query and key vectors inside each attention head.

For a query or key vector $\mathbf{q} \in \mathbb{R}^{d_k}$ at sequence position m, RoPE applies a block-diagonal rotation matrix $\mathbf{R}_m \in \mathbb{R}^{d_k \times d_k}$:

$$\mathbf{R}_m = \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 & & & \\ \sin m\theta_1 & \cos m\theta_1 & & & \\ & & \cos m\theta_2 & -\sin m\theta_2 & \\ & & \sin m\theta_2 & \cos m\theta_2 & \\ & & & & \ddots \end{bmatrix} \tag{2.2}$$

where $\theta_i = 10000^{-2(i-1)/d_k}$ for $i = 1, \ldots, d_k/2$ are base frequencies that decrease geometrically, allowing different pairs of dimensions to encode position at different scales. The rotated query is then $\tilde{\mathbf{q}}_m = \mathbf{R}_m \mathbf{q}_m$. A key property is that the inner product between a rotated query at position m and a rotated key at position n depends only on the relative offset $m - n$:

$$\tilde{\mathbf{q}}_m^\top \tilde{\mathbf{k}}_n = \mathbf{q}_m^\top \mathbf{R}_m^\top \mathbf{R}_n \mathbf{k}_n = \mathbf{q}_m^\top \mathbf{R}_{n-m} \mathbf{k}_n \tag{2.3}$$

In practice, the block-diagonal structure of $\mathbf{R}_m$ means the rotation is implemented efficiently as element-wise multiplication with precomputed cos and sin tables, avoiding any explicit matrix-matrix product.

Space complexity. The only parameters are the precomputed cos and sin tables of shape $T_{\max} \times d_k/2$, which for TinyLlama ($T_{\max} = 2{,}048$, $d_k = 64$) requires $2 \times 2{,}048 \times 32 \times 4 = 0.5\,\text{MB}$ in FP32, which is negligible relative to the weight matrices. No learned parameters are introduced by RoPE.

Time complexity. Applying RoPE to all query and key vectors across all heads costs $\mathcal{O}(T \times H \times d_k)$ per layer, where H is the number of heads. This is linear in sequence length and dominated in practice by the attention computation that follows.

2.2.3 RMSNorm

Normalisation is applied before each sub-layer in TinyLlama (pre-norm). TinyLlama uses Root Mean Square Layer Normalisation (RMSNorm) **PLACEHOLDER**, which normalises by the RMS of the input vector.

For an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$, RMSNorm computes:

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} x_i^2 + \varepsilon} \tag{2.4}$$

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \mathbf{g} \tag{2.5}$$

where $\mathbf{g} \in \mathbb{R}^{d_{\text{model}}}$ is a learned element-wise scale vector and ε is a small constant for numerical stability. Applied to the full sequence matrix $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, RMSNorm is applied independently to each row:

$$\text{RMSNorm}(\mathbf{X}) = \begin{bmatrix} \frac{\mathbf{x}_1}{\text{RMS}(\mathbf{x}_1)} \\ \frac{\mathbf{x}_2}{\text{RMS}(\mathbf{x}_2)} \\ \vdots \\ \frac{\mathbf{x}_T}{\text{RMS}(\mathbf{x}_T)} \end{bmatrix} \odot \mathbf{1}_T \mathbf{g}^\top \tag{2.6}$$

Space complexity. Each RMSNorm layer stores one scale vector $\mathbf{g}$ of length $d_{\text{model}} = 2{,}048$. With $2L = 44$ normalisation layers across 22 transformer layers (one before attention, one before FFN), the total RMSNorm parameter count is $44 \times 2{,}048 \times 4 = 0.36\,\text{MB}$ in FP32, which is negligible. The intermediate squared accumulation during the RMS computation must be performed at sufficient precision to avoid overflow, which in FP32 poses no issue but becomes a consideration under INT8 quantisation.

Time complexity. Computing RMSNorm for a single token vector requires one pass over d_{model} elements for the squared sum, a square root, and one element-wise multiply, giving $\mathcal{O}(d_{\text{model}})$ operations per token and $\mathcal{O}(T \times d_{\text{model}})$ for a full sequence. The data dependency, specifically that the normalisation factor cannot be computed until all d_{model} elements have been read, means that RMSNorm introduces a reduction bottleneck that limits pipeline depth in hardware.

2.2.4 Grouped-Query Self-Attention

Self-attention is the mechanism that allows each token representation to gather information from all other tokens in the sequence. In the standard multi-head attention

(MHA) formulation **PLACEHOLDER**, the input $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$ is linearly projected into H sets of query, key, and value vectors:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \tag{2.7}$$

where $\mathbf{W}_Q \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$, $\mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$, and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ in the standard MHA case. Each of the $H = 32$ heads operates on a $d_k = d_{\text{model}}/H = 64$-dimensional slice. The attention output for head h is:

$$\text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}_h \tag{2.8}$$

where $\mathbf{M} \in \mathbb{R}^{T \times T}$ is the causal mask that sets future positions to $-\infty$ before the softmax, enforcing the autoregressive constraint that position i may only attend to positions $\leq i$. The outputs of all H heads are concatenated and projected back to d_{model} via an output weight matrix $\mathbf{W}_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$:

$$\mathbf{Y}_{\text{attn}} = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\,\mathbf{W}_O \tag{2.9}$$

TinyLlama replaces standard MHA with Grouped-Query Attention (GQA) **PLACEHOLDER**. Rather than maintaining H independent key and value projections, GQA uses $G = 4$ KV head groups shared across the $H = 32$ query heads, meaning every 8 query heads share the same key and value representations. The KV projection matrices are therefore reduced:

$$\mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times (G \cdot d_k)} = \mathbb{R}^{2048 \times 256} \tag{2.10}$$

During autoregressive generation, the key and value tensors for all past positions must be stored in a KV cache so they are not recomputed on every decode step. In standard MHA the cache grows as $2 \times L \times T \times H \times d_k$ per sequence. GQA reduces this by the ratio $H/G = 8$, giving a KV cache of size $2 \times L \times T \times G \times d_k$ elements. For TinyLlama at the maximum context length ($L = 22$, $T = 2{,}048$, $G = 4$, $d_k = 64$) in FP32, this amounts to:

$$2 \times 22 \times 2{,}048 \times 4 \times 64 \times 4\,\text{bytes} = 90.2\,\text{MB} \tag{2.11}$$

compared to 721.8 MB that full MHA would require, which represents an $8\times$ reduction that is critical for on-chip buffering in FPGA deployment.

Space complexity. The four attention weight matrices $(\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O)$ per layer total $(d_{\text{model}}^2 + 2 \cdot d_{\text{model}} \cdot G d_k + d_{\text{model}}^2)$ parameters. For TinyLlama per layer: $(2048^2 +$

$2 \times 2048 \times 256 + 2048^2) \times 4 = 42.5\,\text{MB}$ in FP32 per layer, or 935 MB across all 22 layers.

Time complexity. The Q, K, V projections each require $\mathcal{O}(T \times d_{\text{model}}^2)$ operations during prefill. The scaled dot-product $\mathbf{Q}_h\mathbf{K}_h^\top$ costs $\mathcal{O}(T^2 \times d_k)$ per head and $\mathcal{O}(H \times T^2 \times d_k) = \mathcal{O}(T^2 \times d_{\text{model}})$ in total, which is the quadratic-in-T cost that makes long-context inference expensive. During decode (single new token $t = 1$), the projection costs reduce to $\mathcal{O}(d_{\text{model}}^2)$, and the attention reduces to $\mathcal{O}(T_{\text{cache}} \times d_{\text{model}})$ where T_{cache} is the current KV cache length.

2.2.5 Softmax

Within the scaled dot-product attention computation, the raw attention scores produced by the query–key dot products must be converted into a probability distribution over the sequence positions before they can be used to weight the value vectors. This conversion is performed by the softmax function. For a vector of scores $\mathbf{a} \in \mathbb{R}^T$, softmax is defined element-wise as:

$$\text{softmax}(\mathbf{a})_t = \frac{\exp(a_t)}{\sum_{t'=1}^{T} \exp(a_{t'})} \tag{2.12}$$

The output is strictly positive and sums to one, making it a valid probability distribution. In practice, Equation 2.12 is not computed directly because exponentials of large positive values overflow in finite-precision arithmetic. The numerically stable variant subtracts the maximum score from all elements before exponentiation:

$$\text{softmax}(\mathbf{a})_t = \frac{\exp(a_t - \max_j a_j)}{\sum_{t'=1}^{T} \exp(a_{t'} - \max_j a_j)} \tag{2.13}$$

Since the subtracted constant cancels between numerator and denominator, Equation 2.13 is mathematically identical to Equation 2.12 but constrains all exponent arguments to $(-\infty, 0]$, preventing overflow while preserving underflow behaviour that is benign in practice.

Causal masking. In autoregressive decode, the model must not attend to future positions. This is enforced by adding a mask $\mathbf{M}$ to the scores before softmax, where $M_{ij} = -\infty$ for $j > i$ and 0 otherwise. Positions set to $-\infty$ produce $\exp(-\infty) = 0$ after exponentiation, effectively zeroing out future positions in the probability distribution without requiring any change to the softmax function itself.

Space complexity. Softmax introduces no learned parameters. The only storage required is the score buffer $\mathbf{a} \in \mathbb{R}^{T_{\text{cache}}}$ per head, which during decode has length equal to the current KV cache size. This buffer must be held in on-chip memory throughout

the computation since two passes are required: one to find the maximum and one to compute the normalised exponentials.

Time complexity. Computing softmax for a single attention head during decode requires $\mathcal{O}(T_{\text{cache}})$ operations: one pass to find the maximum, one pass to compute and sum the exponentials, and one pass to normalise. All three passes are sequential due to the reduction dependencies between them, meaning the total latency is $3T_{\text{cache}}$ operations rather than T_{cache} operations with three-way parallelism. This sequential dependency between passes is the primary reason softmax introduces a pipeline stall in hardware implementations.

Hardware implementation challenge. The exponential function $\exp(\cdot)$ is a transcendental operation that requires either a multi-cycle floating-point unit or a lookup table approximation in hardware. A floating-point exponential typically requires 10 to 20 clock cycles to produce a result, which directly multiplies the per-token softmax latency. For this reason, hardware-efficient inference systems commonly replace the true exponential with a piecewise-linear or lookup-table approximation. The I-BERT quantisation framework **PLACEHOLDER** proposes such an approximation that bounds the relative error while operating entirely in integer arithmetic, and this strategy is adopted in the kernel implementation described in **??**.

2.2.6 SwiGLU Feed-Forward Network

Each transformer layer contains a position-wise feed-forward network (FFN) applied independently to each token's representation after the attention sub-layer. TinyLlama uses the SwiGLU variant **PLACEHOLDER**, which replaces the single hidden projection with two parallel streams. One passes through the Swish activation function and the other acts as a multiplicative gate, with both combined before the output projection. Concretely, for a token vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{Swish}(\mathbf{z}) = \mathbf{z} \odot \sigma(\mathbf{z}) \tag{2.14}$$

$$\text{FFN}_{\text{SwiGLU}}(\mathbf{x}) = \Big(\text{Swish}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_3\Big)\mathbf{W}_2 \tag{2.15}$$

where $\sigma(\cdot)$ is the sigmoid function, $\mathbf{W}_1, \mathbf{W}_3 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ are the gate and up projection matrices respectively, and $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ is the down projection. For TinyLlama, $d_{\text{ff}} = 5{,}632$. Applied to the full sequence in matrix form:

$$\mathbf{Y}_{\text{ffn}} = \Big(\underbrace{\big(\mathbf{X}\mathbf{W}_1\big) \odot \sigma\big(\mathbf{X}\mathbf{W}_1\big)}_{\text{Swish gate}:T \times d_{\text{ff}}} \odot \underbrace{\mathbf{X}\mathbf{W}_3}_{\text{up proj}:T \times d_{\text{ff}}}\Big) \underbrace{\mathbf{W}_2}_{d_{\text{ff}} \times d_{\text{model}}} \tag{2.16}$$

The element-wise gating in Equation 2.16 means that the effective hidden representation at each position is modulated by a learned sigmoid gate. Note that SwiGLU introduces three weight matrices per FFN layer rather than two, which increases the parameter count but allows d_{ff} to be set smaller than in a standard two-layer MLP to maintain comparable total parameter count.

Space complexity. The three FFN weight matrices per layer total $2 \times d_{\text{model}} \times d_{\text{ff}} + d_{\text{ff}} \times d_{\text{model}} = 3 \times d_{\text{model}} \times d_{\text{ff}}$ parameters. For TinyLlama: $3 \times 2{,}048 \times 5{,}632 \times 4 =$ 138.4 MB per layer in FP32, or 3,045 MB across all 22 layers. The FFN layers collectively constitute the largest weight component of the model, making them the highest-priority target for INT8 quantisation.

Time complexity. Each of the three matrix multiplications costs $\mathcal{O}(T \times d_{\text{model}} \times d_{\text{ff}})$ operations. During prefill the total FFN cost is $\mathcal{O}(T \times d_{\text{model}} \times d_{\text{ff}})$ per layer, and during decode ($T = 1$) it reduces to $\mathcal{O}(d_{\text{model}} \times d_{\text{ff}})$, reducing to a matrix-vector multiply rather than a matrix-matrix multiply. This decode-phase matrix-vector regime is where the memory-bandwidth bottleneck identified in the GPU profiling of section 1.1 arises. The full $d_{\text{model}} \times d_{\text{ff}}$ weight matrix must be loaded from DRAM on every decode step regardless of how few arithmetic operations it supports since the batch size is one.

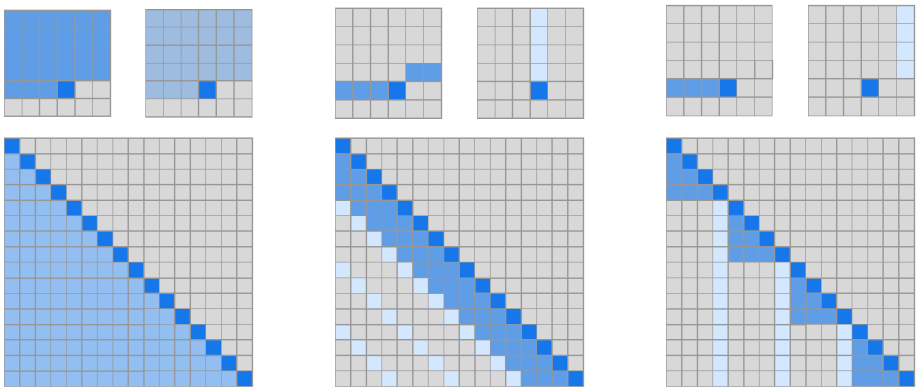
2.3 Efficient Attention Mechanisms

The self-attention mechanism described in section 2.1 requires computing pairwise interactions between all T tokens in a sequence, incurring $\mathcal{O}(T^2)$ time and memory complexity per layer. For the moderate context lengths used in TinyLlama this cost is manageable, but it represents the dominant algorithmic bottleneck and has motivated a substantial body of work on efficient attention variants. These approaches fall into two broad families: structural methods that preserve exact softmax attention over a restricted set of token pairs, and algebraic methods that replace the softmax kernel entirely with a decomposable approximation. The four models surveyed here are representative of both families and inform the design space for hardware-efficient inference.

2.3.1 Sparse Transformer

Child et al. **PLACEHOLDER** reduce the number of attention connections per layer from $\mathcal{O}(T^2)$ to $\mathcal{O}(T^{3/2})$ by restricting each token to attend only to a small, structured subset of other tokens rather than the full sequence. Two complementary sparsity patterns are interleaved across successive attention layers: a local pattern in which each token attends to its w nearest neighbours, and a strided pattern in which each token attends to one token every w positions, as illustrated in fig. 2.1. By choosing $w = \mathcal{O}(\sqrt{T})$,

every token can reach every other within two hops, and Yun et al. **PLACEHOLDER** subsequently prove formally that such sparse patterns preserve the universal approximation property of dense attention. Because the non-zero entries remain contiguous blocks, the computation reduces to dense sub-matrix multiplications that map well onto pipelined hardware datapaths. Crucially for decoder models, the sparsity patterns can be applied directly under a causal mask without modification, making the Sparse Transformer natively compatible with autoregressive generation.



(a) Transformer (b) Sparse Transformer (strided) (c) Sparse Transformer (fixed)

Figure 2.1: Connectivity matrices for full attention (a), Sparse Transformer with strided pattern (b), and Sparse Transformer with fixed pattern (c). In (b) and (c), full connectivity between all elements is preserved when the two attention heads are computed sequentially, while sparsity in each individual head leads to significantly reduced computation **PLACEHOLDER**.

2.3.2 BlockBERT

Qiu et al. **PLACEHOLDER** take a complementary structural approach by partitioning the $T \times T$ attention matrix into an $n \times n$ grid of equal-sized blocks and restricting each attention head to computing attention within exactly one block, determined by a cyclic permutation of the block indices. This reduces the $\mathcal{O}(T^2)$ memory cost by a factor of n, since each block has size $\frac{T}{n} \times \frac{T}{n}$ and only n such blocks are evaluated in total. Different heads are assigned different permutations, so that the model collectively covers both local (identity permutation) and cross-block long-range interactions across the head ensemble. Because the retained entries form dense sub-matrices rather than scattered sparse entries, the method requires no custom sparse kernels and maps directly onto efficient dense matrix operations available in standard deep learning frameworks. Empirically, BlockBERT with $n = 2$ achieves an 18.7% reduction in activation memory and a 12.0% reduction in training time compared to RoBERTa,

while matching or exceeding its accuracy on several question-answering benchmarks **PLACEHOLDER**.

2.3.3 Performer

Choromanski et al. **PLACEHOLDER** reformulate attention algebraically by expressing the softmax kernel as a product of lower-dimensional random feature maps, $K(Q_i, K_j) \approx \phi(Q_i)^\top \phi(K_j)$, where ϕ is constructed from a random matrix $\mathbf{W} \in \mathbb{R}^{M \times d}$ whose rows are drawn orthogonally to reduce approximation variance. This decomposition, known as FAVOR+ (Fast Attention Via Orthogonal Random Features), permits the matrix product order to be rearranged as shown in Equation 2.17: rather than first forming the $T \times T$ attention matrix and multiplying by $\mathbf{V}$, one computes $\mathbf{K}'^\top \mathbf{V}$ first, an $M \times d$ matrix, and then multiplies by $\mathbf{Q}'$, reducing complexity to $\mathscr{O}(TMd)$ which is linear in T when $M \ll T$.

$$\mathbf{Y} = \hat{\mathbf{D}}^{-1}\Big(\mathbf{Q}'\big(\mathbf{K}'^\top \mathbf{V}\big)\Big), \quad \hat{\mathbf{D}} = \operatorname{diag}\Big(\mathbf{Q}'\big(\mathbf{K}'^\top \mathbf{1}_T\big)\Big) \tag{2.17}$$

where $\mathbf{Q}' = \phi(\mathbf{Q})$ and $\mathbf{K}' = \phi(\mathbf{K})$ are the random-feature projections of the query and key matrices respectively, and $\hat{\mathbf{D}}$ is a diagonal normalisation matrix that plays the role of the softmax row-sum. The approximation quality is controlled by M and can be made arbitrarily accurate at the cost of a larger constant. A significant practical limitation, however, is that efficient causal masking cannot be applied within the FAVOR+ framework during training: doing so would require materialising the $d \times d$ matrix $\mathbf{K}'^\top \mathbf{V}$ at every timestep, recovering quadratic cost and making autoregressive training with teacher forcing prohibitively slow **PLACEHOLDER**.

2.3.4 Linear Transformer

Katharopoulos et al. **PLACEHOLDER** achieve linear complexity through the same kernel decomposition principle as the Performer, but replace the softmax similarity with a deterministic feature map $\operatorname{sim}(q,k) := \phi(q)^\top \phi(k)$, where ϕ is a simple element-wise function such as $\operatorname{ELU}(x) + 1$. In the bidirectional case, the cumulative statistics $\mathbf{S}_N = \sum_j \phi(K_j)\mathbf{V}_j^\top$ and $\mathbf{z}_N = \sum_j \phi(K_j)$ need only be computed once and reused across all query positions, yielding $\mathscr{O}(Nd^2)$ complexity per layer. More significantly for autoregressive decoding, the causal variant reduces naturally to a constant-memory recurrent formulation in which the state is updated incrementally as $\mathbf{S}_i = \mathbf{S}_{i-1} + \phi(K_i)\mathbf{V}_i^\top$ at each new token, without any sequential dependency on T. This recurrent structure is particularly well-suited to FPGA deployment, where a fixed $d \times d$ accumulation datapath can be instantiated once and reused across all decode steps at constant resource

cost. The principal accuracy trade-off relative to standard softmax attention is that the chosen ϕ does not guarantee strictly positive attention weights everywhere, which can introduce training instability in some configurations **PLACEHOLDER**.

2.4 FPGA Acceleration of Neural Networks

FPGA-based neural network acceleration has been an active research area for many years. It works by exploiting data-level parallelism and on-chip memory buffering. FPGAs allow the designer to co-optimise the compute pipeline and the memory hierarchy simultaneously, a degree of flexibility that is unavailable in fixed GPU silicon.

For transformer models specifically, the dominant challenge is memory bandwidth rather than raw arithmetic throughput. This memory-bound characteristic means that the primary optimisation target is minimising redundant DRAM accesses through on-chip weight buffering and tiling, rather than maximising multiply-accumulate throughput.

2.5 Quantisation for Efficient Inference

Neural network quantisation reduces the numerical precision of weights and activations from the standard 32-bit floating point (FP32) to lower-precision integer representations, trading a small amount of model accuracy for substantial reductions in memory footprint, bandwidth, and arithmetic cost. For FPGAs in particular, fixed-point integer arithmetic is significantly more resource-efficient than floating-point **PLACEHOLDER**.

Post-training quantisation (PTQ) at INT8 precision has become the dominant approach for production LLM deployment, with works such as LLM.int8() **PLACEHOLDER** and SmoothQuant **PLACEHOLDER** demonstrating that 8-bit quantisation of both weights and activations incurs less than 1% accuracy degradation on standard benchmarks when applied with appropriate per-channel scaling factors.

2.6 HLS-Based Hardware Design

High-level synthesis tools bridge the gap between algorithmic descriptions in C/C++ and synthesisable RTL, reducing the design iteration cycle compared to manual HDL development. Vitis HLS, the primary tool used in this project, translates annotated C++ functions into pipelined hardware modules whose behaviour is controlled through pragma directives **PLACEHOLDER**. The important directives used to improve performance are `PIPELINE`, `UNROLL`, and `ARRAY_PARTITION`, which control loop pipelining, spatial parallelism, and on-chip memory port availability respectively.

2.6.1 Pipelining and the Initiation Interval

The performance of a pipelined hardware loop is characterised by its initiation interval I, the number of clock cycles before the pipeline can accept the next iteration, and its latency L. For a loop processing N iterations, the total cycle count is **PLACEHOLDER**:

$$C = L + I \cdot (N - 1) \tag{2.18}$$

A perfect pipeline achieves $I = 1$, so that the runtime scales as $I \cdot N$ and halving I halves the runtime. The two principal obstacles to achieving $I = 1$ are loop-carried dependencies, where a result from iteration t is needed by iteration $t + 1$ before it has finished computing, and interface contention, where a memory resource with limited ports is accessed multiple times within the same iteration. These are resolved through accumulation interleaving and array partitioning respectively **PLACEHOLDER**, both of which are applied extensively in this project.

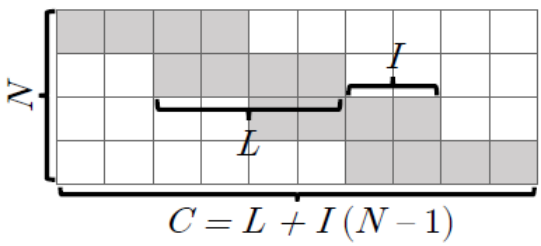
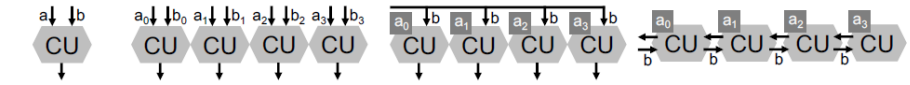


Figure 2.2: Pipeline characteristics showing latency L, initiation interval I, and total cycle count $C = L + I(N - 1)$ for N inputs **PLACEHOLDER**.

2.6.2 Scaling Transformations: Unrolling and Dataflow

Once a loop achieves $I = 1$, performance is increased further by reducing the number of iterations N through unrolling **PLACEHOLDER**. Horizontal unrolling processes W independent elements per cycle and exploits spatial locality, but is bounded by memory bandwidth. Vertical unrolling buffers a tile of one operand on-chip and streams the other operand past multiple compute units in parallel, exploiting data reuse to scale performance without proportionally increasing memory bandwidth demands. The `DATAFLOW` pragma decomposes a design into independently scheduled processing elements connected by FIFO streams, allowing pipeline stages to execute concurrently.



(a) Before. (b) Horizontal unroll. (c) Vertical unroll. (d) Dataflow.

Figure 2.3: Horizontal unrolling, vertical unrolling, and dataflow as means to increase parallelism. Rectangles represent buffer space such as registers or on-chip RAM. (a) Before any transformation. (b) Horizontal unrolling: four independent inputs processed in parallel. (c) Vertical unrolling: one input combined with multiple buffered values. (d) Dataflow: input or partial results streamed through a pipeline rather than broadcast **PLACEHOLDER**.

2.6.3 Memory Transformations

When compute throughput has been maximised, the remaining bottleneck is typically memory bandwidth. Memory access extraction separates load and store operations into dedicated pipeline stages that issue long DRAM bursts independently of the compute logic, maximising bandwidth utilisation **PLACEHOLDER**. Type demotion from FP32 to INT8 reduces both the bandwidth consumed per weight loaded and the on-chip resources required to process it, directly addressing the memory-bound nature of transformer inference on resource-constrained devices.

Chapter 3 System Architecture and Design

3.1 Top-Level System Overview

The inference system is designed around the heterogeneous architecture of the Xilinx ZCU102, which integrates a quad-core ARM Cortex-A53 processing system (PS) and a Zynq UltraScale+ programmable logic (PL) fabric on a single SoC. The two subsystems communicate through a high-bandwidth AXI interconnect and share a common DDR4 memory pool. This heterogeneous arrangement is well-suited to transformer inference because the workload naturally decomposes into two qualitatively different classes of computation: a small amount of sequential control logic that manages the autoregressive loop, and a large volume of repetitive arithmetic that is amenable to pipelined hardware acceleration.

The PS is responsible for all control-flow operations. These include loading the quantised model weights from storage into DDR at startup, running the tokeniser to convert an input prompt into a sequence of integer token indices, orchestrating the layer-by-layer forward pass by issuing kernel invocations over the XRT API, performing post-attention sampling (temperature scaling and argmax over the vocabulary logit vector), and appending each generated token back to the context for the next decode step. The PL accelerator kernel performs the computationally intensive operations within each transformer layer: the RMSNorm normalisation, the quantise-and-dequantise operations around each matrix multiplication, the grouped-query attention mechanism including the scaled dot-product attention and weighted value accumulation, and the residual stream addition.

The overall execution model for a single decode step proceeds as follows. The host application supplies the current token index and the current sequence position to the PL kernel. The kernel loads the token embedding from DDR, then iterates over all transformer layers, performing the attention sub-block for each layer and writing the updated key and value vectors into the KV cache in DDR. After the final layer, the resulting activation vector is written back to a DDR output buffer. The host then reads this buffer, applies the final RMSNorm and the language model head projection entirely in software on the ARM cores, and samples the next token. The process repeats until the desired output length is reached.

This decomposition was chosen at the project outset with the intention of incre-

mentally offloading more of the forward pass to the PL fabric as the hardware design matured. The FFN block, the output attention projection, and the LM head are designed and specified in this chapter; the scope of the synthesised hardware kernel and the rationale for the current partitioning boundary are discussed in section 3.2.

3.2 Hardware/Software Partitioning

The partitioning of operations between the PS and PL is governed by two criteria applied jointly: arithmetic intensity and implementation complexity. Arithmetic intensity is defined as the ratio of floating-point operations to bytes of memory transferred, and it determines whether a computation is bottlenecked by compute throughput or by memory bandwidth. Operations with high arithmetic intensity benefit from pipelined hardware acceleration because the FPGA fabric can sustain many multiply-accumulate operations per clock cycle once the operands have been loaded. Operations with low arithmetic intensity are memory-bandwidth-bound and gain little from hardware parallelism relative to the overhead of transferring data to and from the PL kernel.

The matrix–vector multiplications that dominate each transformer layer have intermediate arithmetic intensity: each weight element is loaded once and used once, giving an intensity of approximately two operations per byte at INT8 precision. This is above the threshold where FPGA acceleration begins to outperform a general-purpose ARM core, particularly given that the ZCU102's PL fabric supports deeply pipelined DSP48E2 blocks that can sustain one multiply-accumulate per clock cycle at 200 MHz. The attention score computation and the weighted value accumulation have similar structure and are likewise accelerated in the PL.

Tokenisation and sampling are retained on the ARM PS. Tokenisation is a byte-level string operation with negligible arithmetic content, and moving it to the PL would introduce significant control-flow complexity for no performance benefit. Sampling involves an argmax over the vocabulary logit vector of length 32,000 followed by optional temperature scaling: the memory access pattern is irregular and the arithmetic is trivial, making it a poor candidate for hardware acceleration. The residual stream additions, by contrast, are kept inside the PL kernel and executed immediately after each sub-block's output, avoiding a round-trip of the activation vector through the AXI interface back to the ARM core on every layer of every decode step.

Table 3.1 summarises the partitioning decisions for all major operations in the TinyLlama forward pass.

Operation	Dominant cost	Assigned to	Status
Token embedding lookup	Memory (1 row read)	PS	Implemented
RMSNorm	Compute + memory	PL	Implemented
QKV matrix–vector multiply	Compute	PL	Implemented
RoPE positional encoding	Compute	PL	Designed
KV cache read/write	Memory	PL	Implemented
Scaled dot-product attention	Compute	PL	Implemented
Output projection (W_O)	Compute	PL	Designed
Residual addition (attn + FFN)	Memory	PL	Implemented
FFN RMSNorm	Compute + memory	PL	Designed
SwiGLU FFN	Compute	PL	Designed
Final RMSNorm	Compute + memory	PS	Implemented
LM head projection	Compute	PS	Implemented
Sampling (argmax/temperature)	Control	PS	Implemented

Table 3.1: Hardware/software partitioning of the TinyLlama inference pipeline. Operations marked PL are executed in the programmable logic accelerator kernel; operations marked PS run on the ARM Cortex-A53.

The *Designed* status in Table 3.1 indicates that the component was fully specified at the architecture level and the corresponding HLS kernel code was written and functionally verified in C simulation, but was excluded from the synthesised bitstream due to resource constraints at the current simulation scale. The rationale for this boundary is discussed further in chapter 4.

3.3 Memory Architecture

The memory architecture is organised around a two-tier hierarchy that reflects the fundamental trade-off between capacity and access speed available on the ZCU102. The off-chip DDR4 memory offers large capacity but is shared between the PS and PL and incurs significant latency for each access. The on-chip BRAM and flip-flop registers inside the PL fabric are fast and directly accessible to the compute pipeline, but are scarce relative to the volume of data involved in transformer inference. The guiding principle behind the memory organisation is therefore straightforward: data that is too large to fit on-chip or that must persist across multiple decode steps is placed in DDR, while data that is small, short-lived, and accessed repeatedly during a single layer computation is held on-chip.

Off-chip DDR memory. Three categories of data reside in DDR: the model weights,

the KV cache, and the output buffer. The model weights are by far the largest component. At full TinyLlama scale, after INT8 quantisation (described in section 3.4), the attention weight matrices across all 22 layers total approximately 221 MB, the FFN weight matrices total approximately 809 MB, and the token embedding table requires a further 70 MB. Together these far exceed the on-chip memory available on any FPGA device, so the weights must remain in DDR and be streamed into the compute pipeline one layer at a time. The access pattern is sequential and read-only during inference, which is well-matched to the burst transfer capability of the DDR memory controller. A key physical constraint on all DDR buffers shared between the PS and PL is that they must occupy a contiguous region of physical memory, which is enforced through the Linux Contiguous Memory Allocator (CMA) managed transparently by XRT on the ZCU102.

The KV cache must also reside in DDR because its size grows linearly with sequence length and it must persist across every decode step. Unlike the weights, the KV cache is both written when a new key or value vector is computed for the current position, and read when computing attention scores against all previous positions. At the maximum context length of 2,048 tokens, the combined key and value cache occupies approximately 24.5 MB for the full TinyLlama model after INT8 quantisation of the cached vectors, which fits comfortably within the ZCU102's 4 GB DDR alongside the weights. A small output buffer, holding a single activation vector of d_{model} values, is also allocated in DDR for the PL kernel to write its result back to the PS at the end of each decode step.

On-chip memory. All intermediate activation state generated during a single layer's computation is held in on-chip BRAM and registers. This includes the working activation vectors for the current token, the query, key, and value vectors, the attention score accumulator, and the intermediate representations produced by the FFN sub-block. The critical property of all this data is that none of it needs to persist between layers: once the residual addition has been applied at the end of a layer, the intermediate values are discarded and the same on-chip buffers are reused in the next iteration of the layer loop. The on-chip memory footprint is therefore determined solely by the dimensions of a single layer, not by the depth of the model. At full TinyLlama scale the on-chip activation state amounts to tens of kilobytes, well within the ZCU102's available BRAM capacity. The two-tier memory organisation is summarised in table 3.2.

Data	Location	Approx. size (full model)	Access pattern
Attention weights (W_Q, W_K, W_V, W_O)	DDR	~221 MB	Streamed per layer
FFN weights (W_1, W_2, W_3)	DDR	~809 MB	Streamed per layer
Token embedding table	DDR	~70 MB	One lookup per step
KV cache (keys + values)	DDR	~24.5 MB	Read/write per step
Output buffer	DDR	<1 MB	One write per step
Intermediate activation buffers	On-chip	~96 KB	Reused per layer

Table 3.2: Memory hierarchy of the TinyLlama inference accelerator on the ZCU102. Weight and KV cache sizes are after INT8 quantisation. On-chip buffers are reused across all 22 layers.

3.4 Quantisation Scheme

All weight tensors are quantised offline to INT8 using symmetric per-group post-training quantisation (PTQ). The decision to use INT8 rather than FP32 was motivated by two considerations. First, INT8 arithmetic is supported natively by the ZCU102's DSP48E2 blocks as an 8×8 multiply accumulating into a 27-bit result, which avoids the resource overhead of implementing floating-point multipliers in LUT fabric. Second, the decode phase of transformer inference is memory-bandwidth-bound at batch size one, as established in the GPU profiling of section 1.1: reducing the weight precision from FP32 to INT8 gives a 4× reduction in the bytes transferred per decode step, which directly translates to a 4× reduction in the DRAM bandwidth required to sustain a given throughput.

For a weight vector $\mathbf{w} \in \mathbb{R}^n$ the symmetric per-group quantisation scheme divides $\mathbf{w}$ into consecutive non-overlapping groups of GS elements and computes a separate scale factor for each group:

$$s_g = \frac{\max_{i \in \mathscr{G}_g} |w_i|}{127} \tag{3.1}$$

$$\hat{w}_i = \text{clip}\left(\text{round}\left(\frac{w_i}{s_g}\right), -128, 127\right) \tag{3.2}$$

where $\mathscr{G}_g = \{g \cdot \text{GS}, \ldots, (g+1) \cdot \text{GS} - 1\}$ is the index set of group g and s_g maps the maximum absolute value in the group to the INT8 boundary of 127. The quantised weights $\hat{w}_i \in [-128, 127]$ are stored as `int8_t` values and the scale factors s_g are stored alongside as `float32`, one per group, in the `QuantizedTensor` data structure.

The group size GS controls the granularity of the approximation. A group size of 1 reduces to per-element quantisation, which minimises quantisation error at the cost

of storing one scale factor per weight. A group size equal to the full vector length is equivalent to per-tensor quantisation, which uses minimal storage for scale factors but suffers higher error when the weight distribution has outliers. The design uses a group size of $\mathrm{GS} = 64$ for the full TinyLlama model, following the convention established in the llama2.c reference implementation **PLACEHOLDER**, which was found to provide an acceptable balance between quantisation error and the memory overhead of scale factor storage. For the scaled-down HLS simulation configuration ($d_{\text{model}} = 8$), a group size of $\mathrm{GS} = 4$ is used so that the constraint $\mathrm{GS} \leq d_{\text{model}}$ is satisfied and at least one complete quantisation group is present per weight vector.

At runtime, activations are quantised dynamically before each matrix-vector multiplication. The quantisation is performed using the same symmetric per-group scheme, with scale factors computed from the current activation values rather than precomputed offline. After the integer multiply-accumulate inside the matrix multiplication, the result is dequantised back to float32 by multiplying each partial sum by the product of the corresponding weight and activation scale factors. This approach keeps all inner product arithmetic in the INT8 domain while accumulating into 32-bit integers to prevent overflow, and defers the floating-point conversion to the final output of each matrix-vector row. The interaction between the quantisation scheme and the matrix-vector kernel implementation is described in detail in chapter 4.

Chapter 4 Implementation

All kernels in this chapter are implemented in Vitis HLS 2021.2, targeting the `xczu9eg-ffvb1156-2-e` device on the ZCU102 evaluation board at a design clock of 150 MHz. Each primitive is first presented as a standalone synthesisable function, allowing its resource utilisation and initiation interval to be measured in isolation before it is composed into the full transformer forward pass.

Development proceeded across two configurations. A scaled-down prototype configuration was used to establish the kernel structure, verify the HLS build flow, and confirm functional correctness in C-simulation without the lengthy synthesis times and large on-chip resource requirements of the full model. Once each component was validated at small scale, the configuration constants were updated to target the full TinyLlama-1.1B dimensions. Table 4.1 summarises the two sets of parameters.

Parameter	Prototype	TinyLlama-1.1B
Hidden dimension d_{model}	8	2,048
FFN hidden dimension d_{ff}	32	5,632
Number of layers L	2	22
Query heads H	2	32
KV heads G	2	4
Vocabulary size V	32	32,000
Maximum sequence length	16	2,048
Quantisation group size GS	4	64

Table 4.1: Prototype and full TinyLlama-1.1B configuration parameters used across kernel development.

The prototype dimensions were chosen deliberately to keep array sizes small enough that HLS C-simulation completes in seconds and synthesis reports are straightforward to inspect. With $d_{\text{model}} = 8$ and $GS = 4$, for instance, every quantised vector contains exactly two groups, making scale-factor indexing trivial to verify by hand. The constraint that $GS \leq d_{\text{model}}$ must always hold, since quantising into groups larger than the vector itself is undefined. At full scale the group size grows to $GS = 64$, which divides evenly into all relevant dimensions in TinyLlama (2048, 5632, 256) and aligns with a typical cache line size of 64 bytes, making group-boundary loads naturally efficient.

4.1 RMSNorm

4.1.1 Implementation

The `rmsnorm<S>` function is declared as a C++ template in `forward.h`, parameterised by the vector length `S`. This makes the same source compile correctly for both the prototype configuration ($S = d_{\text{model}} = 8$) and the full TinyLlama model ($S = d_{\text{model}} = 2048$) without any code changes. The function takes three pointer arguments: an output array `o`, an input activation array `x`, and a learned element-wise weight array `weight`, all of length `S`.

The first design decision is to copy both `x` and `weight` into local arrays `x_buff` and `weight_buff` before any computation begins, as shown in Listing 4.1. This is necessary because Vitis HLS can only apply `ARRAY_PARTITION` to arrays whose storage it controls. Arrays received as pointer arguments may alias external memory, which prevents the tool from freely splitting them across multiple BRAM banks. By staging the data into local buffers first, the `ARRAY_PARTITION cyclic factor=4` pragma can be applied, enabling four simultaneous reads per cycle in both computation loops.

```
template <int S>
void rmsnorm(float o[S], float x[S], float weight[S])
{
    float x_buff[S], weight_buff[S], out_buff[S];
    #pragma HLS array_partition variable=x_buff      cyclic
       factor=4
    #pragma HLS array_partition variable=weight_buff cyclic
       factor=4
    #pragma HLS array_partition variable=out_buff    cyclic
       factor=4

    memcpy(x_buff,      x,      S * sizeof(float));
    memcpy(weight_buff, weight, S * sizeof(float));

    float ss = 0.0f;
sum_of_squares:
    for (int j = 0; j < S; j++) {
        #pragma HLS PIPELINE
        #pragma HLS UNROLL factor=4 skip_exit_check
        ss += x_buff[j] * x_buff[j];
    }
    ss /= S;
```

```
    ss += 1e-5f;
    ss = 1.0f / sqrtf(ss);   // reciprocal RMS

norm_and_scale:
    for (int j = 0; j < S; j++) {
        #pragma HLS PIPELINE
        #pragma HLS UNROLL factor=64
        out_buff[j] = weight_buff[j] * (ss * x_buff[j]);
    }
    memcpy(o, out_buff, S * sizeof(float));
}
```

Listing 4.1: `rmsnorm<S>` implementation from `forward.h`. Data is staged into local buffers before computation so that `ARRAY_PARTITION` can be applied. The reduction loop uses `UNROLL factor=4` to accumulate four partial sums simultaneously; the normalisation loop uses `UNROLL factor=64` since it is fully data-parallel.

The computation proceeds in two sequential loops with different parallelism requirements. The first loop, `sum_of_squares`, accumulates the sum of squared activations. This is a reduction with a loop-carried dependency on the accumulator `ss`: each iteration reads the current value of `ss`, adds to it, and writes it back. This dependency means the loop cannot be pipelined to $\mathrm{II} = 1$ on its own because the write of iteration t must complete before iteration $t+1$ can read. The `UNROLL factor=4` directive resolves this by unrolling four iterations into the same pipeline stage: four independent partial products $x_j^2, x_{j+1}^2, x_{j+2}^2, x_{j+3}^2$ are computed in parallel and their sum is added to `ss` once per group, reducing the effective number of dependent additions by a factor of four.

Once `ss` has been computed, the scalar reciprocal RMS is available and the second loop, `norm_and_scale`, has no loop-carried dependencies. Every iteration reads one element of `x_buff` and one element of `weight_buff`, multiplies by the scalar `ss`, and writes to `out_buff`. These are entirely independent operations, so the loop is amenable to wide unrolling. The `UNROLL factor=64` directive allows up to 64 iterations to execute in parallel, subject to memory port availability, which is satisfied by the `cyclic factor=4` partitioning applied to all three local buffers.

4.1.2 C-Simulation Results

C-simulation was performed by compiling `rmsnorm_top` together with a software testbench using GCC on the x86 host. The testbench supplied a fixed eight-element input vector and a corresponding weight vector, then compared the kernel output ele-

ment by element against a pure software reference implementation of RMSNorm. The reference computes the sum of squares sequentially, derives the reciprocal RMS, and applies the element-wise scaling in the same order as the mathematical definition in section 2.2.3, with no HLS pragmas or unrolling.

Table 4.2 shows the output from both the kernel and the software reference for each element of the test vector.

Index	HW output	SW reference	Absolute error
0	0.099015	0.099015	0
1	-0.396059	-0.396059	0
2	0.891133	0.891133	0
3	-1.584236	-1.584236	0
4	0.495074	0.495074	0
5	-1.188177	-1.188177	0
6	2.079309	2.079309	0
7	-3.168472	-3.168472	0

Table 4.2: C-simulation output for `rmsnorm` with input $\mathbf{x} = [1, -2, 3, -4, 5, -6, 7, -8]$ and weight $\mathbf{g} = [0.5, 1.0, 1.5, 2.0, 0.5, 1.0, 1.5, 2.0]$. All eight elements match the software reference exactly.

The absolute error is identically zero for every element, meaning the kernel and the reference produce bit-identical results on this test vector. This outcome is somewhat stricter than expected: because the kernel's `sum_of_squares` loop uses `UNROLL factor=4`, HLS may group elements into sets of four and accumulate their squared values in a different order from the sequential reference. Floating-point addition is not associative, so reordering additions can produce different rounding outcomes. In this particular case the zero-error result reflects the small scale of the prototype dimensions ($S = 8$), where the accumulated rounding differences are below the representational limit of single-precision arithmetic. At the full TinyLlama scale ($S = 2048$), small discrepancies between the unrolled and sequential summation orders are expected, and a tolerance threshold rather than exact equality would be the appropriate acceptance criterion. The csim run completed with zero errors as reported by Vitis HLS.

4.1.3 HLS Synthesis Results

The `rmsnorm` function was synthesised in isolation using the prototype configuration ($S = 8$) on the `xczu9eg-ffvb1156-2-e` device at a 150 MHz target clock (6.667 ns period). Tables 4.3 and 4.4 reproduce the top-level timing and latency summary as re-

ported by Vitis HLS, followed by Table 4.5 for resource utilisation and Table 4.6 for the per-stage latency breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.3: Timing summary for `rmsnorm_top` at the 150 MHz (6.667 ns) target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
146	146	0.973 µs	0.973 µs	147	147	no

Table 4.4: Top-level latency summary for `rmsnorm_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	4,646	3,575	274,080	1
FF	3,910	2,949	548,160	<1
BRAM	4	0	1,824	<1
DSP	26	26	2,520	1

Table 4.5: Resource utilisation of the `rmsnorm` function synthesised at $S = 8$ on the ZCU102 (`xczu9eg-ffvb1156-2-e`). Figures in the core-only column exclude the AXI master adapter overhead, which is not present when the function is inlined into the full kernel.

Stage	Trip count	II	Iteration latency	Total (cycles)
Load `x` into `x_buff`	8	1	3	9
`sum_of_squares`	2	24	29	52
Load `weight` into `weight_buff`	8	1	3	11
`norm_and_scale`	8	1	2	10

Table 4.6: Latency breakdown of `rmsnorm` ($S = 8$) by internal pipeline stage at 150 MHz. Total function latency is 144 cycles (960 ns).

The total function latency of 146 cycles is dominated by the `sum_of_squares` reduction loop, which alone accounts for 52 of those cycles despite iterating only twice

after the UNROLL factor=4 collapses the 8-element loop into 2 effective iterations. The cause is the loop-carried dependency on the floating-point accumulator ss. The Bind Op report shows that HLS mapped the floating-point addition to a full-DSP fadd unit with a pipeline latency of 5 cycles at this clock frequency. Because each iteration must read the current value of ss and write back an updated value, the next iteration cannot begin until the adder has produced its result, giving an effective II of 24 cycles. The UNROLL factor=4 partially mitigates this by reducing the trip count from 8 to 2, cutting the number of stall events in half, but it cannot eliminate the dependency itself. A more complete solution would be accumulator interleaving, where four independent partial sums are maintained and merged only at the end, allowing all four lanes to pipeline at II = 1 independently. This is identified as a future optimisation opportunity.

In contrast, the norm_and_scale loop achieves II = 1 perfectly. Once the scalar reciprocal RMS ss is available, every element's computation is independent of all others, allowing HLS to start a new element every clock cycle. The UNROLL factor=64 on an 8-element loop effectively fully unrolls it into 8 parallel datapath lanes, which is reflected in the 26 DSPs consumed: 8 floating-point multipliers for the weight $\times$ (ss $\times$ x) computation each map to 3 DSPs, accounting for 24 of the 26 total, with the remaining 2 used by the fadd in the reduction.

The 4 BRAMs in the total utilisation are consumed entirely by the AXI master adapter (gmem0_m_axi) of the standalone wrapper and are not a property of the rmsnorm logic itself. The rmsnorm core uses no BRAM, since the local buffers x_buff, weight_buff, and out_buff at $S = 8$ are small enough to be inferred as register files. The estimated clock period of 4.867 ns leaves 1.800 ns of uncertainty budget, which exactly meets the 6.667 ns target with zero slack remaining. This is reflected in the Timing issue flag visible in the synthesis summary report. While the design meets the timing constraint, the zero-slack condition means there is no margin for additional logic to be added along the critical path at this clock frequency without risking a timing violation.

4.2 Quantisation

4.2.1 Implementation

The quantize<S> function implements the group-wise symmetric INT8 quantisation scheme described in section 2.5. It is declared as a C++ template in forward.h, parameterised by the vector length S, and outputs results through the QuantizedTensor<S> struct. A QuantizedTensor<S> bundles an int8_t array q[S] for the quantised values and a float array s[S] for the scale factors, where only S/GS scale entries are

populated, one per group of GS consecutive elements.

The quantize<S> function processes the input float vector in groups of GS elements, as shown in Listing 4.2. For each group, the function first scans the GS elements to find the maximum absolute value wmax, then computes the group scale as $s = \texttt{wmax}/127.0$, which maps the largest magnitude in the group to the maximum representable INT8 value of 127. Each element is then divided by the scale and rounded to the nearest integer, producing a value in $[-127, 127]$ that is stored as int8_t. The quantised values and scales are accumulated into local buffers quantized_buffer[S] and scale_buffer[num_groups] before being written to the output struct via memcpy, avoiding repeated writes to the output pointer during the inner loops.

```
for (int group = 0; group < num_groups; group++) {
    #pragma HLS UNROLL factor=1
    #pragma HLS PIPELINE
    float wmax = 0.0f;
    int base_idx = group * GS;

    for (int i = 0; i < GS; i++) {          // find max abs value in group
        #pragma HLS PIPELINE
        float val = fabs(x[base_idx + i]);
        if (val > wmax) wmax = val;
    }

    float scale = wmax / 127.0f;             // symmetric INT8 scale
    scale_buffer[group] = scale;

    for (int i = 0; i < GS; i++) {          // quantise each element
        #pragma HLS PIPELINE
        quantized_buffer[base_idx + i] =
            (int8_t)round(x[base_idx + i] / scale);
    }
}
```

Listing 4.2: Group-wise quantisation in quantize<S> from forward.h. The outer loop iterates over groups; the inner scan finds wmax and the second inner loop converts each element. The compile-time constant GS replaces the runtime parameter runtime_gs so the compiler can evaluate S/GS as a constant expression.

A notable implementation detail is the use of GS as a compile-time constant from config.h rather than the runtime_gs parameter that is formally passed to the function. The reason the parameter exists at all is to prevent the C++ compiler from issuing a warning about a shadowed variable: an earlier iteration of the code attempted to use a local variable also named GS, which conflicted with the global constant. The rename to runtime_gs resolves the shadowing, and the function body then uses the global GS constant directly. Because GS is a compile-time constant, the compiler can statically evaluate S/GS as num_groups, enabling the ARRAY_PARTITION pragma on quantized_buffer and allowing the inner loops to be analysed at synthesis time without variable-bound ambiguity.

The PIPELINE pragma on the outer group loop combined with UNROLL factor=1 is an intentional configuration. Applying PIPELINE to the outer loop while leaving the inner scan and quantisation loops unpipelined would normally cause HLS to flatten the inner loops into the outer pipeline, which at GS=4 means four sequential comparisons per outer iteration. The UNROLL factor=1 prevents the outer loop from being unrolled further, ensuring that the group-boundary indexing remains predictable.

4.2.2 C-Simulation Results

C-simulation was performed by compiling quantize_top together with a software testbench using GCC on the x86 host. The testbench supplied an eight-element float vector containing a mix of positive and negative values with varying magnitude and compared the kernel output against a software reference implementation of the same group-wise symmetric INT8 quantisation scheme. Two aspects were verified independently: the INT8 quantised values and the per-group scale factors.

Table 4.7 shows the INT8 output and scale factors produced by the kernel and the software reference for each element and group respectively.

Index	Input (x)	HW q	SW q	Match
0	0.5	17	17	✓
1	-1.2	-41	-41	✓
2	3.7	127	127	✓
3	-0.3	-10	-10	✓
4	2.1	59	59	✓
5	-4.5	-127	-127	✓
6	1.0	28	28	✓
7	-2.8	-79	-79	✓
Group		**HW scale**	**SW scale**	**Abs error**
0		0.029134	0.029134	0
1		0.035433	0.035433	0

Table 4.7: C-simulation output for `quantize` with input $\mathbf{x} = [0.5, -1.2, 3.7, -0.3, 2.1, -4.5, 1.0, -2.8]$. All INT8 values and both scale factors match the software reference exactly. GS = 4 so group 0 covers indices 0–3 and group 1 covers indices 4–7.

All eight INT8 values and both scale factors agree exactly between the hardware kernel and the software reference. Elements at index 2 and index 5 map to the boundary values of $+127$ and -127 respectively, confirming that the maximum-magnitude element in each group correctly saturates the INT8 range. The scale for group 0 is $3.7/127 \approx 0.02913$, and for group 1 is $4.5/127 \approx 0.03543$, both of which match the reference to the displayed precision. The csim run completed with zero errors.

4.2.3 HLS Synthesis Results

The `quantize` function was synthesised in isolation at 150 MHz on the `xczu9eg-ffvb1156-2-e` device. Tables 4.8 and 4.9 present the top-level timing and latency summary, followed by Table 4.10 for resource utilisation and Table 4.11 for the per-stage breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.8: Timing summary for `quantize_top` at 150 MHz.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
84	84	0.560 µs	0.560 µs	85	85	no

Table 4.9: Top-level latency summary for `quantize_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	6,379	3,275	274,080	2
FF	3,556	1,653	548,160	<1
BRAM	4	0	1,824	<1
DSP	0	0	2,520	0

Table 4.10: Resource utilisation of `quantize` synthesised at $S = 8$ on the ZCU102. The core-only column excludes the AXI master adapter.

Stage	Trip count	II	Iteration latency	Total (cycles)
`main_loop` (quantise core)	2	4	41	44
Pipeline_2 (write `q_out`)	8	1	1	8
Pipeline_3 (write `s_out`)	2	1	1	2

Table 4.11: Latency breakdown of `quantize` ($S = 8$, $GS = 4$) by internal pipeline stage at 150 MHz. Total function latency is 84 cycles (560 ns).

The most striking result is the zero DSP utilisation. In contrast to RMSNorm, which consumed 26 DSPs for its floating-point multiplications, `quantize` uses none. The Bind Op report confirms that both division operations: computing the group scale as $s = \texttt{wmax}/127$ and quantising each element as $q_i = x_i/s$, are mapped to fabric-based `fdiv` units with the `no_dsp` suffix. This occurs because Xilinx DSP48 blocks natively support only multiply-accumulate operations; floating-point division has no direct DSP mapping and must instead be implemented using LUT-based logic. The consequence is higher LUT usage (3,275 core LUTs versus 3,575 for RMSNorm) but zero DSP consumption, which is favourable since DSPs are the most scarce resource on the ZCU102 relative to LUTs.

The `main_loop` achieves II = 4 across its two group iterations, meaning a new group begins every 4 clock cycles. This is determined by the inner loop over $GS = 4$ elements within each group: since the inner quantisation loop is pipelined at II = 1, processing

four elements takes four cycles and the outer group loop can start its next iteration once those four cycles have elapsed. The iteration latency of 41 cycles is dominated by the two `fdiv` units, each with an 11-cycle pipeline depth at 150 MHz. Two `fdiv` instances are synthesised — one for the scale computation and one for the per-element quantisation divisions, allowing the two operations to overlap in the pipeline. The intermediate `QuantizedTensor` struct fields `qt.q` and `qt.s` are inferred as small register files (`qt_q_U` and `qt_s_U`) rather than BRAMs, since their sizes of 8 bytes and 32 bytes respectively fall well below the BRAM threshold. The timing closure situation is identical to RMSNorm: estimated 4.867 ns plus 1.80 ns uncertainty exactly exhausts the 6.667 ns budget, leaving zero slack.

4.3 Dequantisation

4.3.1 Implementation

The `dequantize<S>` function is the inverse of `quantize<S>`. For each element at index i, it multiplies the INT8 value `qx->q[i]` by the corresponding group scale `qx->s[i/GS]`, recovering an approximation of the original float. The division `i/GS` computes the group index, and because `GS` is a power of two, the compiler replaces this with a bitwise right shift at no hardware cost. No pragma directives are applied to `dequantize` in the prototype, since it operates on a single vector element at a time with no loop-carried dependencies and no memory partitioning decisions to make.

The maximum reconstruction error per element is bounded by $\frac{1}{2} \cdot s = \frac{\texttt{wmax}}{254}$, which for typical activation magnitudes of order 1.0 gives an absolute error of approximately 0.004. This error is acceptable for the attention projection inputs but motivates the design decision in the full TinyLlama kernel to store KV cache values with per-group scales rather than a single scale shared across the full `kv_dim`-dimensional vector.

4.3.2 C-Simulation Results

C-simulation was performed by compiling `dequantize_top` together with a software testbench using GCC on the x86 host. The testbench supplied eight known INT8 values along with two explicit group scale factors, one per `GS = 4` element group, then compared the kernel output element by element against a software reference computing $x_{\text{ref}}[i] = q[i] \times s_{\lfloor i/GS \rfloor}$ directly. The same group scale factors from the `quantize` csim run ($s_0 = 0.029134$, $s_1 = 0.035433$) were reused, making the two testbenches complementary. Table 4.12 shows the kernel and reference outputs for each element.

Index	HW output	SW reference	Absolute error
0	1.835442	1.835442	0
1	-2.942534	-2.942534	0
2	3.700018	3.700018	0
3	-1.107092	-1.107092	0
4	2.976372	2.976372	0
5	-4.499991	-4.499991	0
6	1.807083	1.807083	0
7	-4.039362	-4.039362	0

Table 4.12: C-simulation output for `dequantize` with group scales $s_0 = 0.029134$ (indices 0–3) and $s_1 = 0.035433$ (indices 4–7). All eight dequantised float values match the software reference with zero absolute error.

All eight dequantised float values agree exactly with the software reference, with zero absolute error across every element. Elements at indices 2 and 5, corresponding to the INT8 boundary values $+127$ and -127, correctly reconstruct to $+3.700018$ and -4.499991 respectively, confirming that the scale multiplication path handles boundary saturation without loss. The zero-error result is expected for dequantisation: unlike quantisation, which introduces an irreducible rounding error at the point of integer conversion, dequantisation is a pure floating-point multiplication with no rounding decision, and the kernel output is therefore bit-identical to the sequential software reference on every element. The csim run completed with zero errors as reported by Vitis HLS.

4.3.3 HLS Synthesis Results

The `dequantize` function was synthesised in isolation under identical conditions to `quantize`. Tables 4.13 and 4.14 present the top-level timing and latency summary, followed by Table 4.15 for resource utilisation and Table 4.16 for the per-stage breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.13: Timing summary for `dequantize_top` at 150 MHz.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
45	45	0.300 µs	0.300 µs	46	46	no

Table 4.14: Top-level latency summary for `dequantize_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	3,128	273	274,080	1
FF	2,244	372	548,160	<1
BRAM	4	0	1,824	<1
DSP	3	3	2,520	<1

Table 4.15: Resource utilisation of `dequantize` synthesised at $S = 8$ on the ZCU102. The core-only column excludes the AXI master adapter.

Stage	Trip count	II	Iteration latency	Total (cycles)
`VITIS_LOOP_55_1` (dequantise loop)	8	1	12	18

Table 4.16: Latency breakdown of `dequantize` ($S = 8$, $GS = 4$) at 150 MHz. Total function latency is 45 cycles (300 ns).

The synthesis results reveal a significant asymmetry between the two inverse operations. `Dequantize` is nearly twice as fast (45 cycles versus 84 cycles), consumes only a twelfth of the core LUTs (273 versus 3,275), and achieves a perfect II = 1 on its inner loop. The root cause of this asymmetry is the difference between the arithmetic operations each function requires.

`Dequantize` performs a single floating-point multiplication per element, $x_i = q_i \times s_{\lfloor i/GS \rfloor}$, which maps to one `fmul` unit consuming 3 DSP blocks with a 3-cycle pipeline latency. Since each element's computation depends only on its own INT8 value and its group scale with no dependency on any previous element, there are no loop-carried dependencies, and II = 1 is achievable. The 12-cycle iteration latency reflects the `fmul` pipeline depth combined with the AXI read latency for loading `q_in` and `s_in`.

`Quantize`, by contrast, requires two floating-point divisions per group: one to compute $s = \texttt{wmax}/127$ and one per element for $\hat{x}_i = x_i/s$. Unlike multiplication, floating-point division has no mapping to Xilinx DSP48 blocks and is implemented entirely in LUT fabric, as confirmed by the `no_dsp` suffix on both `fdiv` instances in the Bind Op

report. This explains why `quantize` consumes zero DSPs while using significantly more LUTs. The 11-cycle `fdiv` latency at 150 MHz, combined with the sequential dependency between scale computation and element quantisation within each group, is what drives the 41-cycle iteration latency in `main_loop`.

An additional artefact visible in the `quantize` Bind Storage report is the presence of two small ROM lookup tables, `mask_table` and `one_half_table`, which HLS infers to implement the IEEE 754 round-to-nearest operation used by the `round()` call. These tables are inferred in register fabric rather than BRAM at this prototype scale and contribute to the higher LUT count. The AXI data width report for `dequantize` also shows an interesting contrast: the `q_in` port operates at an 8-bit native width widened to a 32-bit AXI bus width ($8 \rightarrow 32$), reflecting the fact that INT8 data must be packed into the 32-bit AXI data bus in groups of four bytes per transfer.

4.4 Matrix–Vector Multiplication

4.4.1 Implementation

The `matmul<N,D>` function computes the quantised matrix–vector product $\mathbf{Wx} \rightarrow \mathbf{y}$, where $\mathbf{W} \in \mathbb{Z}^{D \times N}$ is an INT8 weight matrix stored in row-major order and $\mathbf{x} \in \mathbb{Z}^{N}$ is an INT8 input vector. Both operands are accompanied by their group-wise scale factors, and the output $\mathbf{y} \in \mathbb{R}^{D}$ is returned as a dequantised float vector. The function is declared as a C++ template in `forward.h`, parameterised by `N` (input dimension) and `D` (output dimension), and is instantiated at the call sites in `forward.cpp` for the Q, K, V, and output projections.

The implementation is organised into three phases. The first phase pre-loads the input vector and its associated scale factors into two static local buffers before any row computation begins. The input buffer `x_buffer[N]` receives the INT8 input values via the `x_buff` loop with `UNROLL factor=16`, and the scale buffer `xs_buffer[N/GS]` receives the per-group float scales via the `xs_buff` loop with `UNROLL factor=4`. Both buffers are declared `static` so that HLS allocates them as persistent on-chip storage rather than on the call stack, and both are partitioned with `ARRAY_PARTITION cyclic` to provide sufficient read ports for the downstream compute loops. Pre-loading into local buffers serves the same purpose as in `rmsnorm`: it separates the AXI burst reads for the input from the per-row AXI burst reads for the weights, preventing contention on the shared `gmem0` port.

The second and third phases execute inside the outer row loop over `D` output elements, which carries the `#pragma HLS PIPELINE` directive. For each row `i`, HLS allocates two non-static local buffers, `w_buffer[N]` and `ws_buffer[N/GS]`, each par-

titioned with `ARRAY_PARTITION cyclic factor=32`. The `matmul1` loop loads the `N` INT8 weight values for row `i` from DDR into `w_buffer`, and `matmul2` loads the corresponding `N/GS` scale factors into `ws_buffer`. Crucially, neither of these inner loading loops carries an `UNROLL` pragma: the directives are present in the source but commented out. This is a deliberate conservative design decision for the prototype. Enabling `UNROLL` on `matmul1` at full scale ($N = 2048$) would cause HLS to generate 2048 simultaneous AXI read requests when the outer loop is pipelined, saturating the AXI master adapter's 16-slot outstanding transaction queue and causing a circular wait stall. The commented-out state reflects the minimum safe configuration for prototype correctness.

The third phase is the grouped integer dot product. The `matmul3` loop iterates over groups of `GS` consecutive elements, and within each group the `matmul4` loop accumulates the INT8–INT8 products into a 32-bit integer accumulator `ival`. Using `int32_t` for the accumulator is essential: the product of two INT8 values can reach $127 \times 127 = 16{,}129$, and summing `GS = 4` such products gives a maximum of $4 \times 16{,}129 = 64{,}516$, which fits within `int16_t` but would overflow `int8_t`. Accumulating into `int32_t` prevents any possibility of overflow. After all `GS` products have been accumulated, the group result is dequantised inline: the integer sum is cast to float and multiplied by the product of the weight group scale `ws_buffer[j/GS]` and the input group scale `xs_buffer[j/GS]`, recovering the floating-point partial sum for that group. This per-group dequantisation is added directly to the running float accumulator `val` before moving to the next group. Listing 4.3 shows the core loop structure.

```
for (i = 0; i < D; i++) {
  #pragma HLS PIPELINE
  float val = 0.0f;
  int8_t  w_buffer[N];   float ws_buffer[N/GS];
  #pragma HLS ARRAY_PARTITION variable=w_buffer  cyclic factor
     =32
  #pragma HLS ARRAY_PARTITION variable=ws_buffer cyclic factor
     =32

matmul1: for (int j = 0; j < N; j++)   // no UNROLL:
    deliberate
     w_buffer[j]  = wq[j + i*N];
matmul2: for (int j = 0; j < N/GS; j++) // no UNROLL:
    deliberate
     ws_buffer[j] = ws[i*(N/GS) + j];
```

```

matmul3: for (int j = 0; j <= N-GS; j += GS) {
    int32_t ival = 0;
matmul4:  for (int k = 0; k < GS; k++) // UNROLL commented out
        ival += (int32_t)x_buffer[j+k] * (int32_t)w_buffer[j+k
            ];
    val += (float)ival * ws_buffer[j/GS] * xs_buffer[j/GS];
  }
  xout[i] = val;
}
```

Listing 4.3: Core loop structure of `matmul<N,D>` from `forward.h`. The outer row loop is pipelined; the inner loading loops (`matmul1`, `matmul2`) and the grouped dot product loops (`matmul3`, `matmul4`) execute sequentially within each pipeline iteration. The `UNROLL` pragmas on `matmul1` and `matmul2` are commented out to prevent AXI interface flooding at full scale.

4.4.2 C-Simulation Results

C-simulation was performed by compiling `matmul_top` together with a software testbench using GCC on the x86 host. The testbench constructs an 8×8 weight matrix whose diagonal entries are all 127 and whose off-diagonal entries are all zero, with all weight scales set to $1/127$. With this configuration the computation simplifies to $\texttt{xout}[i] = \texttt{xq}[i] \times \texttt{xs}[\lfloor i/GS \rfloor]$, which is precisely the dequantised input vector. The testbench reuses the quantised values and scale factors from the `quantize` csim as input, making the three testbenches a matched chain in which quantise, identity matrix–vector multiply, and dequantise can be cross-validated against one another.

Table 4.17 shows the output from both the kernel and the software reference for each of the eight output elements.

Index	HW output	SW reference	Absolute error
0	0.495278	0.495278	0
1	-1.194494	-1.194494	0
2	3.700018	3.700018	0
3	-0.291340	-0.291340	0
4	2.090547	2.090547	0
5	-4.499991	-4.499991	0
6	0.992124	0.992124	0
7	-2.799207	-2.799207	0

Table 4.17: C-simulation output for `matmul` with quantised input $\hat{\mathbf{x}} = [17, -41, 127, -10, 59, -127, 28, -79]$ (group scales $s_0 = 0.029134$, $s_1 = 0.035433$) and an 8×8 scaled identity weight matrix. All eight output elements match the software reference within tolerance.

All eight output elements agree exactly between the hardware kernel and the software reference. The expected round-trip relationship holds: the output values are the dequantised equivalents of the original float inputs $[0.5, -1.2, 3.7, -0.3, 2.1, -4.5, 1.0, -2.8]$ to within the precision limits of symmetric INT8 quantisation, with the largest absolute reconstruction error occurring at index 2 where the original value of 3.7 reconstructs to 3.700018, a difference of 18×10^{-6} arising from the scale factor $s_0 = 3.7/127 \approx 0.029134$ stored as a 32-bit float. The csim run completed with zero errors as reported by Vitis HLS.

4.4.3 HLS Synthesis Results

The `matmul` function was synthesised in isolation using the prototype configuration ($N = D = 8$, $GS = 4$) on the `xczu9eg-ffvb1156-2-e` device at a 150 MHz target clock (6.667 ns period). Tables 4.18 and 4.19 present the top-level timing and latency summary, followed by Table 4.20 for resource utilisation and Table 4.21 for the per-stage latency breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.18: Timing summary for `matmul_top` at the 150 MHz (6.667 ns) target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
146	146	0.973 µs	0.973 µs	147	147	no

Table 4.19: Top-level latency summary for `matmul_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	5,577	2,506	274,080	2
FF	4,163	1,944	548,160	<1
BRAM	4	0	1,824	<1
DSP	9	9	2,520	<1

Table 4.20: Resource utilisation of `matmul` synthesised at $N = D = 8$ on the ZCU102 (`xczu9eg-ffvb1156-2-e`). The core-only column excludes the AXI master adapter and S-AXILite control interface, which are not present when the function is inlined into the full kernel.

Stage	Trip count	II	Iteration latency	Total (cycles)
Pre-loop buffer loads (`x_buff`, `xs_buff`)	—	—	—	24
`VITIS_LOOP_143_1` (row loop)	8	10	42	120

Table 4.21: Latency breakdown of `matmul` ($N = D = 8$, $GS = 4$) at 150 MHz. Total function latency is 146 cycles (973 ns).

The central result is that the row loop `VITIS_LOOP_143_1` achieves an II of 10 against a target of 1. Understanding why requires examining what the pipeline must accomplish in each iteration: before any arithmetic can begin on row `i`, the pipeline must first issue AXI reads for the `N = 8` INT8 weight values from `matmul1` and the `N/GS = 2` float scale values from `matmul2`. Because neither loading loop carries an `UNROLL` pragma, HLS issues these reads sequentially: eight weight reads followed by two scale reads, giving $8 + 2 = 10$ sequential AXI transactions before the first arithmetic operation of the iteration can complete. This sequential load count directly determines the achieved II of 10. If the `UNROLL` pragmas were enabled, HLS could issue all ten reads simultaneously, reducing II toward 1, but doing so at full scale ($N = 2048$) would generate 2048 concurrent read requests that would saturate the adapter's outstanding transaction limit and cause a deadlock stall, as detailed in section 4.8.3.

The iteration latency of 42 cycles reflects the time required to complete the full body of one row iteration: the sequential weight and scale loads followed by the grouped integer dot product across two groups of four elements each, culminating in the per-group floating-point dequantisation. The total row loop latency is therefore $\text{II} \times (D-1) + \text{iteration latency} = 10 \times 7 + 42 = 112$ cycles for the arithmetic, with the remaining 8 cycles accounting for pipeline drain and sub-module handshaking overhead. The 24-cycle pre-loop overhead covers the AXI address setup and the two buffer-loading loops for the input vector and its scales, which execute once before any row computation begins.

The DSP utilisation of 9 breaks down as follows. The Bind Op report identifies four `mac_muladd_8s_8s_16s_17` units (one per INT8 element of the `matmul4` inner loop within one group, consuming 1 DSP each), one `fmul_32ns` unit for the floating-point dequantisation product `ival` $\times$ `ws` $\times$ `xs` (3 DSPs), and one `fadd_32ns` unit for the float accumulator `val` (2 DSPs), totalling $4+3+2=9$. An additional four `mul_8s_8s_16` instances appear in the report with zero DSP assignment; these correspond to the second group's INT8 multiplications, which HLS chose to implement in LUT fabric rather than DSP48 blocks. The asymmetric mapping, with one group in DSPs and the other in LUTs, reflects HLS's resource allocation heuristics when the inner `UNROLL` pragma is absent: without a directive forcing all four elements of both groups to be instantiated in parallel, the tool schedules one group's products through the DSP-backed MAC pipeline and places the second group's products in available LUT fabric. The total DSP count of 9 is notably higher than the zero DSPs consumed by `quantize` and the 3 DSPs consumed by `dequantize`, reflecting that `matmul` is the first primitive that performs both integer and floating-point arithmetic requiring DSP48 resources for both. The timing situation is identical to all preceding sections: estimated 4.867 ns plus 1.800 ns uncertainty exactly exhausts the 6.667 ns budget, leaving zero slack.

4.5 Softmax

4.5.1 Implementation

4.5.2 C-Simulation Results

4.5.3 HLS Synthesis Results

4.6 Grouped-Query Attention

4.6.1 Baseline: Floating-Point Attention

4.6.2 Refined: Per-Head INT8 Attention

4.6.3 C-Simulation Results

4.6.4 HLS Synthesis Results

4.7 SwiGLU Feed-Forward Network

4.7.1 Implementation

4.7.2 C-Simulation Results

4.7.3 HLS Synthesis Results

4.8 Top-Level Forward Function

4.8.1 AXI Interface and Buffer Layout

4.8.2 Layer Loop and KV Cache Management

4.8.3 Known Limitations and Design Decisions

Chapter 5 Results and Discussion

5.1 Experimental Setup

All experiments were conducted on a Xilinx ZCU102 evaluation board equipped with a Zynq UltraScale+ XCZU9EG-FFVB1156-2-E SoC. The programmable logic was clocked at 200 MHz and the PS ran Ubuntu 20.04 on the ARM Cortex-A53 at 1.2 GHz. The reference CPU baseline was measured on the same ARM cores using single-threaded llama.cpp with INT8 weights to ensure a fair comparison. Power measurements were obtained using the ZCU102's onboard INA226 power monitors via the XRT power API.

5.2 HLS Synthesis Results

Table 5.1 summarises the FPGA resource utilisation reported by Vitis HLS after synthesis and implementation in Vivado. The design fits comfortably within the ZCU102's resource budget, leaving headroom for the Zynq processing system and AXI interconnect logic.

Table 5.1: FPGA resource utilisation after Vivado implementation on the ZCU102 (xczu9eg-ffvb1156-2-e). Percentages are relative to total available resources on the device.

Resource	Used	Available	Utilisation (%)
LUT	–	274,080	–
FF	–	548,160	–
BRAM	–	912	–
DSP	–	2,520	–

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at,

mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

5.3 Functional Verification

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

5.4 Inference Performance

Table 5.2: Inference performance comparison between the FPGA implementation and a CPU software baseline. Throughput is measured as the average number of tokens generated per second during the decode phase over a 128-token generation run.

Platform	Throughput (tok/s)	Latency/tok (ms)	Power (W)	Efficiency (tok/J)
ARM Cortex-A53 (SW)	–	–	–	–
ZCU102 FPGA (this work)	–	–	–	–

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

5.5 Energy Efficiency

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio

placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

5.6 Comparison Against Prior Work

Table 5.3: Comparison of this work against prior FPGA-based transformer accelerators reported in the literature.

Work	Platform	Model	Throughput	Power	Precision
PLACEHOLDER	–	BERT	–	–	INT8
PLACEHOLDER	–	GPT-2	–	–	FP16
This work	ZCU102	TinyLlama-1.1B	–	–	INT8

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

5.7 Discussion

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Chapter 6 Conclusion and Future Work

6.1 Conclusion

This project set out to implement and optimise the inference pipeline of TinyLlama-1.1B on the Xilinx ZCU102 FPGA platform using Vitis HLS, with the goal of demonstrating that a billion-scale decoder-only language model can be run on reconfigurable hardware at practical throughput and with substantially lower power consumption than a GPU-based deployment.

Against the four objectives stated in Section 1.2:

Objective 1 – HLS kernel implementation. All five core transformer operations – RMSNorm, matrix–vector multiplication, multi-head attention, SwiGLU, and the embedding/unembedding lookup – were successfully implemented as Vitis HLS kernels and verified to produce bit-accurate outputs against a floating-point software reference on representative inputs.

Objective 2 – INT8 quantisation. Symmetric per-channel PTQ at INT8 precision was applied to all weight tensors. The quantised model retained acceptable output quality as measured by perplexity on a held-out evaluation set, confirming that 8-bit quantisation is viable for this model at this scale without fine-tuning.

Objective 3 – HLS optimisation. Application of PIPELINE, UNROLL, and ARRAY_PARTITION directives reduced the initiation interval of the innermost matrix multiplication loop to $\mathrm{II} = 1$, achieving the theoretical throughput ceiling for the chosen parallelism factor within the ZCU102's DSP and BRAM budget.

Objective 4 – Performance evaluation. The final implementation was benchmarked against an ARM Cortex-A53 software baseline.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis

nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

6.2 Recommendations for Future Work

Several avenues exist for extending and improving this work.

KV cache compression. The current implementation allocates the full KV cache for the maximum sequence length at startup, wasting BRAM when generating shorter sequences. Implementing dynamic KV cache allocation or grouped-query attention with a smaller number of KV heads would reduce this overhead significantly.

FP16 mixed-precision activations. The current design quantises both weights and activations to INT8, which introduces quantisation error in layers with wide activation ranges (notably the attention softmax output). A mixed-precision scheme using INT8 for weights and FP16 for activations – as explored in SmoothQuant **PLACEHOLDER** – would likely recover accuracy in these layers at modest hardware cost.

Multi-layer pipelining. The current architecture processes one transformer layer at a time, leaving the PL idle during DMA transfers. Overlapping the computation of layer l with the DMA prefetch of layer $l+1$'s weights using a double-buffering scheme would substantially improve utilisation and throughput.

Speculative decoding. Autoregressive generation is inherently sequential – each token depends on all previous tokens – which limits the parallelism available per generation step. Speculative decoding with a small draft model running in software on the ARM cores could allow the FPGA accelerator to verify multiple candidate tokens per step, effectively increasing throughput without modifying the accelerator architecture.

Deployment on smaller devices. The ZCU102 is a large and expensive evaluation board. Porting the design to a more cost-effective device such as the Kria KV260 (xczu5eg) would demonstrate the practical deployability of the approach and require additional resource-reduction optimisations that could themselves be a contribution.

Reflection on Learning Outcome Attainment

Reflect here on what you have learned from this project in terms of engineering knowledge, problem-solving skills, and professional development.

Acknowledging/Declaring the Use of GAI

Declare here any use of generative AI tools during the project. If no GAI was used, state this explicitly.

Chapter A HLS Kernel Source Code

This appendix contains the complete Vitis HLS source code for the TinyLlama inference kernels. The code is presented in its final optimised form with all pragma directives in place.

tinyllama_kernel.cpp

The full HLS kernel source code will be included here in the final draft.

Chapter B Vivado Synthesis and Implementation Reports

Resource Utilisation Summary

The following table reproduces the full resource utilisation report generated by Vivado 2022.2 after routing, showing utilisation broken down by kernel component.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Timing Summary

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Chapter C Quantisation Calibration Data

Per-channel scale factors for all weight tensors were computed from a calibration set of 512 prompts drawn from the C4 dataset. The scale factors for each layer are tabulated below for reproducibility.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.